PROJECT REPORT

House Price Prediction System

An End-to-End Machine Learning Web Application (FastAPI + Streamlit)

Prepared by: Souraneel Biswas

Executive Summary

This project implements an end-to-end machine learning web application for estimating residential property prices from structured property characteristics. The system follows a two-tier architecture: a FastAPI backend that serves the trained model via REST API endpoints, and a Streamlit frontend that provides an interactive browser-based user interface.

The project contains a 1,000-row CSV dataset, a model-training pipeline (train_model.py), persisted model artefacts in the model/ directory, and a FastAPI application (app.py) paired with a Streamlit frontend (frontend.py). The training pipeline compares three regression algorithms — Linear Regression, Random Forest, and Gradient Boosting — and selects the best by R^2 score.

The saved evaluation artefacts show that Linear Regression is the selected model, with an R^2 of 0.9984, MAE of $8,174.58, and RMSE of $10,071.48. The result is consistent with the dataset's strong linear relationship between Square_Footage and House_Price.

The Streamlit frontend is organised into three tabs: Prediction, Model Metrics, and Data Explorer. The frontend communicates with the FastAPI backend via HTTP REST calls. Plotly is used for interactive visualisations including a price gauge, scatter plots, histograms, and a correlation heatmap. The trained model is persisted as a scikit-learn Pipeline (including the StandardScaler) using joblib.

Key Outcomes

• A complete training-to-deployment workflow is implemented in Python across two runtime components.

• Three regression algorithms are trained and evaluated on the same held-out test set.

• Linear Regression provides the strongest test metrics, selected automatically by the training script.

• The FastAPI backend exposes three REST endpoints: health check, model info, and prediction.

• The Streamlit frontend calls the backend API and renders an interactive prediction interface with gauge, metrics, and data exploration tabs.

• The project is suitable as an academic demonstration of supervised regression, REST API design, and ML application deployment.

1. Introduction and Background

House price estimation is a supervised regression problem in which historical property observations are used to learn a mapping from property characteristics to a continuous target value. A machine learning system can automate this estimation and provide a repeatable, consistent calculation.

This project delivers that regression system as a two-tier web application. The model-training script (train_model.py) prepares the dataset, splits it into training and test subsets, trains three candidate regressors within a scikit-learn Pipeline (including StandardScaler), evaluates them on MAE, RMSE, and R^2, and serialises the best model along with feature metadata to the model/ directory. The FastAPI application (app.py) then loads those artefacts at startup and exposes them via REST endpoints. The Streamlit frontend (frontend.py) consumes those endpoints and presents the prediction interface in a browser.

1.1 Project Motivation

The main motivation is to demonstrate how a trained model can be served as a REST API and consumed by a frontend application — the standard pattern for production ML deployment. Instead of a single monolithic Streamlit script, the project separates concerns: the backend owns the model and inference logic, while the frontend owns the user experience.

1.2 Project Character

• Supervised learning: the target variable House_Price is known during training.

• Regression: the output is a continuous monetary value.

• Batch training: train_model.py trains models offline and stores artefacts.

• REST API inference: the FastAPI backend performs predictions in response to HTTP POST requests.

• Interactive frontend: the Streamlit application calls the API and renders results interactively.

2. Problem Statement

Given a property's physical and neighbourhood characteristics, estimate its house price using a trained regression model served via a REST API, and present the estimate in an interactive web interface.

2.1 Input Variables

The application accepts seven property attributes: Square_Footage, Num_Bedrooms, Num_Bathrooms, Year_Built, Lot_Size, Garage_Size, and Neighborhood_Quality. These seven features are used directly as model inputs — no additional feature engineering is performed at inference time.

2.2 Output

The primary output is an estimated price in US dollars, returned as JSON from the /predict endpoint and displayed prominently in the Streamlit UI. Supporting visuals include a price gauge comparing the prediction to the dataset average, an input summary, and dataset feature range cards.

3. Objectives

1. Build a reproducible regression training pipeline using the project dataset.

2. Train and compare three regression algorithms under a common evaluation protocol.

3. Select and persist the strongest model (as a scikit-learn Pipeline) using joblib.

4. Serve the model via a FastAPI backend with documented REST endpoints.

5. Develop a Streamlit frontend that consumes the API and provides prediction and analytics tabs.

6. Provide interactive visualisations for price gauges, scatter plots, histograms, and correlation heatmaps.

7. Document the system for academic presentation, maintenance, and future extension.

4. Scope and Use Cases

The current implementation is an academic prototype demonstrating the REST API deployment pattern for machine learning rather than a production real-estate valuation service.

4.1 In Scope

• Structured property-price prediction from the supplied CSV dataset.

• Comparison of Linear Regression, Random Forest, and Gradient Boosting.

• StandardScaler embedded in a scikit-learn Pipeline within the saved model artefact.

• FastAPI backend with /health, /model-info, and /predict endpoints.

• Streamlit frontend with Prediction, Model Metrics, and Data Explorer tabs.

• Interactive Plotly visualisations: gauge, scatter, histogram, heatmap, bar charts.

• Local execution with two terminal processes (uvicorn + streamlit).

4.2 Out of Scope

• Live property listings, geospatial data, interest rates, taxes, or real-time market conditions.

• Location-level valuation based on actual addresses.

• Authentication, multi-user access control, database persistence, or production monitoring.

• Statistically calibrated prediction intervals.

5. Technology Stack and Requirements

Technology	Role in the Project
Python 3.11	Core programming language for data processing, modelling, API, and frontend logic.
FastAPI	REST API backend; serves model predictions and metadata via HTTP endpoints.
Uvicorn	ASGI server that runs the FastAPI application.
Pydantic	Request/response validation and serialisation for the FastAPI endpoints.
Streamlit	Interactive web frontend; consumes the FastAPI backend via HTTP.

Technology	Role in the Project
pandas	CSV loading, tabular manipulation, and feature DataFrame construction.
NumPy	Numerical operations and metric calculations.
scikit-learn	Pipeline, StandardScaler, LinearRegression, RandomForest, GradientBoosting, evaluation metrics.
joblib	Persistence of the trained Pipeline model.
Plotly	Interactive gauge, scatter, bar, histogram, and heatmap visualisations in the frontend.
Matplotlib / Seaborn	Static feature importance plot saved to plots/ at training time.
requests	HTTP client used by the Streamlit frontend to call the FastAPI backend.

5.1 Dependency Versions

Package	Pinned Version
fastapi	0.111.0
uvicorn[standard]	0.29.0
pydantic	2.7.1
scikit-learn	1.4.2
pandas	2.2.2
numpy	1.26.4
joblib	1.4.2
streamlit	1.35.0
requests	2.32.2
plotly	5.22.0
matplotlib	3.9.0
seaborn	0.13.2

5.2 Hardware / Software Baseline

- Operating system: Windows, macOS or Linux with Python 3.11+.
- Python 3.11 (confirmed on the target machine).
- RAM: 4 GB or more is sufficient for this 1,000-row dataset.
- Two terminal windows: one for uvicorn (backend) and one for streamlit (frontend).
- Browser: any modern browser for the Streamlit UI at http://localhost:8501.

6. System Architecture

The system follows a two-tier client-server architecture. The training pipeline creates persistent artefacts; the FastAPI backend loads those artefacts at startup and serves predictions via REST; the Streamlit frontend calls the backend over HTTP and renders results interactively.

6.1 Processing Flow

Training phase: train_model.py loads the CSV, splits data 80/20, trains three Pipeline models (StandardScaler + regressor), evaluates each on the held-out test set, selects the best by R^2, saves model.pkl and meta.json to model/, and generates feature_importance.png in plots/.

Inference phase: The user fills the sidebar form in the Streamlit frontend and clicks Predict Price. The frontend sends a JSON POST request to http://localhost:8000/predict. The FastAPI backend validates the request via Pydantic, constructs a one-row DataFrame in the saved feature order, calls model.predict() (the Pipeline handles scaling internally), and returns the predicted price as JSON. The frontend renders the price, gauge, and input summary.

6.2 Major Components

Layer	Main File / Artefact	Responsibility
Data Layer	house_price_regression_dataset.csv	Stores the 1,000 property observations.
Training Layer	train_model.py	Loads CSV, splits data, trains 3 Pipeline models, evaluates and saves best model.
Model Layer	model/model.pkl, model/meta.json	Trained Pipeline (scaler + regressor) and feature/metrics metadata.
API Layer	app.py	FastAPI backend: loads artefacts, validates requests, returns JSON predictions.
Application Layer	frontend.py	Streamlit frontend: calls API, renders prediction, metrics, and data explorer tabs.

7. Dataset Description

The supplied CSV (house_price_regression_dataset.csv) contains 1,000 rows and 8 columns: seven predictors and one target. All columns are numeric. The dataset contains no missing values and no duplicate rows.

Column	Type	Observed Range	Description
Square_Footage	Numeric / integer	503 – 4,999 sq ft	Living area of the property.
Num_Bedrooms	Integer	1 – 5	Number of bedrooms.
Num_Bathrooms	Integer	1 – 3	Number of bathrooms.
Year_Built	Integer	1950 – 2022	Original construction year.
Lot_Size	Numeric / float	0.506 – 4.989 acres	Land area.
Garage_Size	Integer	0 – 2 cars	Garage capacity.
Neighborhood_Quality	Integer	1 – 10	Project-defined neighbourhood quality score.
House_Price	Numeric / float	$111,626 – $1,108,237	Target variable in USD.

7.1 Dataset Summary Statistics

Measure	Value	Measure	Value
Rows	1,000	Missing values	0
Duplicate rows	0	Source columns	8 (7 predictors + 1 target)
Average neighbourhood quality	5.615 / 10	Average garage size	1.022 cars
Average bedrooms	2.99	Average bathrooms	1.973

7.2 Feature Statistics from meta.json

Feature	Min	Mean	Max
Square_Footage	503.0	2,815.4 sq ft	4,999.0
Num_Bedrooms	1.0	2.99	5.0
Num_Bathrooms	1.0	1.97	3.0
Year_Built	1950	1986.6	2022
Lot_Size	0.506 acres	2.778 acres	4.989 acres
Garage_Size	0	1.022	2
Neighborhood_Quality	1	5.615	10

8. Data Preparation

This project uses the seven raw CSV features directly as model inputs. No additional feature engineering (e.g. House_Age, Total_Rooms, Sqft_Per_Room) is performed in train_model.py. This is an intentional design decision that keeps the inference pipeline simple: the saved feature list in meta.json exactly matches the seven source columns, and the FastAPI endpoint validates the same seven fields via Pydantic.

8.1 Train/Test Split

The script uses train_test_split with test_size=0.2 and random_state=42. Therefore 800 observations are used for training and 200 for the held-out test set. The fixed random state makes the split reproducible.

8.2 Scaling

StandardScaler is embedded as the first step of a scikit-learn Pipeline for the Linear Regression model. The scaler is fitted on the training data only and applied to both training and test inputs during pipeline evaluation. Because the scaler is part of the Pipeline object saved in model.pkl, no separate scaler artefact is needed — model.predict() handles preprocessing automatically.

Tree-based models (RandomForest, GradientBoosting) are trained without a scaler since decision-tree-based algorithms are scale-invariant.

9. Machine Learning Methodology

Three regression algorithms are trained and compared. The purpose is to select the best-performing model automatically rather than assuming a fixed choice.

Model	Role / Principle	Configuration
Linear Regression	Fits a linear relationship between features and price. Wrapped in a Pipeline with StandardScaler.	Default hyperparameters
Random Forest Regressor	Ensemble of decision trees; aggregates predictions. Scale-invariant.	n_estimators=200, random_state=42
Gradient Boosting Regressor	Additive sequence of weak learners reducing residual error. Scale-invariant.	n_estimators=300, learning_rate=0.05, max_depth=5, random_state=42

9.1 Evaluation Metrics

Metric	Full Name	Formula	Interpretation
MAE	Mean Absolute Error	$(1/n)\ \Sigma\ \|y_i - \hat{y}_i\|$	Lower is better; average absolute price error in USD.
RMSE	Root Mean Squared Error	$\sqrt{[(1/n)\ \Sigma(y_i - \hat{y}_i)^2]}$	Lower is better; penalises larger errors more strongly.
R^2	Coefficient of Determination	$1 - SS_{res}/SS_{tot}$	Higher is better; proportion of target variance explained.

10. Model Training and Evaluation Results

The following test-set metrics are taken directly from model/meta.json (all_model_metrics), which is the artefact consumed by app.py and exposed via the /model-info endpoint.

Model	MAE	RMSE	R^2
LinearRegression (selected)	$8,174.58	$10,071.48	0.9984
GradientBoosting	$12,195.85	$14,880.71	0.9966
RandomForest	$16,035.17	$19,678.14	0.9940
Source	model/meta.json → all_model_metrics		

10.1 Selected Model

The training script automatically selects the model with the highest R^2 score. In this run, Linear Regression (within a StandardScaler Pipeline) achieves R^2 = 0.9984, MAE = $8,174.58, and RMSE = $10,071.48. Relative to the mean dataset price of approximately $618,861, the MAE is approximately 1.32% of the mean price — an excellent fit for this dataset.

The selected model is saved as model/model.pkl (a Pipeline object) and its name is stored in model/meta.json as "model_name": "LinearRegression". The FastAPI backend reads this name and returns it with every prediction response via the model_used field.

11. FastAPI Backend Design

The backend (app.py) is implemented with FastAPI and serves the trained model via three endpoints. At startup, it loads model/model.pkl and model/meta.json. If model.pkl is not found, it raises a RuntimeError with an instructive message to run train_model.py first.

CORS is configured with allow_origins=["*"] so the Streamlit frontend (running on a different port) can call the API without cross-origin restrictions.

11.1 API Endpoints

Method	Path	Description
GET	/	Health check — returns {"status": "ok"}.
GET	/model-info	Returns model name, metrics, all model metrics, feature list, and feature stats.
POST	/predict	Accepts HouseFeatures JSON body; returns predicted_price, model_used, currency.

11.2 Request / Response Schema

The POST /predict endpoint accepts a JSON body matching the HouseFeatures Pydantic model with the following fields and validation rules:

Field	Type	Validation
Square_Footage	float	gt=0 (must be positive)
Num_Bedrooms	int	ge=1, le=10
Num_Bathrooms	int	ge=1, le=10
Year_Built	int	ge=1800, le=2024
Lot_Size	float	gt=0 (must be positive)
Garage_Size	int	ge=0, le=5
Neighborhood_Quality	int	ge=1, le=10

The response is a PredictionResponse object containing predicted_price (float), model_used (string, e.g. "LinearRegression"), and currency (string, always "USD").

Pydantic validation rejects out-of-range inputs automatically (e.g. Year_Built outside 1800–2024, negative Square_Footage), returning a 422 Unprocessable Entity response with details. Internal model errors return a 500 response.

Interactive API documentation is available at http://localhost:8000/docs (Swagger UI) when the backend is running.

12. Streamlit Frontend Design

The frontend (frontend.py) is a Streamlit application that communicates with the FastAPI backend via the requests library. It is structured around three tabs: Prediction, Model Metrics, and Data Explorer.

12.1 Sidebar — Input Form

The sidebar contains seven input controls for the seven model features. When the user clicks the Predict Price button, the frontend assembles the inputs into a JSON payload and sends a POST request to http://localhost:8000/predict. If the backend is unreachable, an instructive error message is shown.

Control	Type	Range / Options
Square Footage	Slider	500 – 5,000 sq ft, step 50, default 2,000
Bedrooms	Selectbox	1 – 5, default 3
Bathrooms	Selectbox	1 – 5, default 2
Year Built	Slider	1900 – 2024, step 1, default 2000
Lot Size (acres)	Slider	0.1 – 10.0, step 0.1, default 2.0
Garage Capacity	Selectbox	0 – 5 cars, default 1
Neighborhood Quality	Slider	1 – 10, step 1, default 5

12.2 Tab 1 — Prediction

• Displays the predicted price in a prominent styled price card with the model name below it.

• Shows a 7-field Input Summary using st.metric widgets.

• Renders an interactive Plotly gauge (Indicator) comparing the predicted price to the dataset average ($618,861), with colour bands for low / mid / high price ranges.

• Before prediction, shows Dataset Feature Ranges cards displaying mean, min, and max for each feature from the /model-info API response.

12.3 Tab 2 — Model Metrics

• Displays the full model comparison table (all three models) sourced from the /model-info endpoint, sorted by R^2 descending, with best-row highlighting.

• R^2 bar chart (Plotly Express, Blues colour scale) showing R^2 by model.

• Grouped MAE vs RMSE bar chart (Plotly Graph Objects) for direct error comparison.

• KPI metrics for the selected model: MAE, RMSE, and R^2 displayed with st.metric.

12.4 Tab 3 — Data Explorer

• Interactive scatter plot of Square_Footage vs House_Price, with a user-selectable colour dimension (Neighborhood_Quality, Num_Bedrooms, Num_Bathrooms, or Garage_Size) and point size proportional to Lot_Size.

• Price distribution histogram (40 bins).

• Correlation heatmap (px.imshow, RdBu_r colour scale) across all numeric features.

• Raw data expander showing the first 100 rows of the CSV.

13. Sample Prediction Walkthrough

The following example shows a representative prediction using the default sidebar values. The frontend sends these values as a JSON POST to /predict, and the backend returns the estimated price.

Input / Output	Value
Square Footage (default)	2,000 sq ft
Bedrooms	3
Bathrooms	2
Year Built	2000
Lot Size	2.0 acres
Garage	1 car
Neighbourhood Quality	5 / 10
Model Used (response)	LinearRegression
Currency (response)	USD

Prediction Sequence:

1. User adjusts sliders and dropdowns in the Streamlit sidebar.

2. User clicks Predict Price.

3. frontend.py assembles a JSON payload with the seven feature values.

4. HTTP POST to http://localhost:8000/predict.

5. app.py validates the payload via Pydantic, constructs a one-row DataFrame in the saved feature order.

6. model.predict() is called — the Pipeline applies StandardScaler then LinearRegression internally.

7. JSON response: {"predicted_price": <value>, "model_used": "LinearRegression", "currency": "USD"}.

8. Frontend renders price card, gauge, and input summary.

14. Testing and Validation

The project does not contain a dedicated automated unit-test suite. Validation is based on direct inspection of the training code, saved artefacts, API behaviour, and the running application.

Test Area	Test	Observed Result	Status
Dataset integrity	Load CSV and inspect shape	1,000 × 8; no missing values; 0 duplicates	Pass
Model persistence	Load model/model.pkl and model/meta.json	Artefacts present and loadable; Pipeline structure verified	Pass
Backend startup	uvicorn app:app --reload	Starts on http://127.0.0.1:8000; startup complete message confirmed	Pass
Health endpoint	GET /	Returns {"status": "ok", "message": "House Price Prediction API is running."}	Pass
Model info endpoint	GET /model-info	Returns model_name, metrics, all_model_metrics, features, feature_stats	Pass
Prediction endpoint	POST /predict with valid payload	Returns predicted_price, model_used='LinearRegression', currency='USD'	Pass
Input validation	POST /predict with invalid Year_Built=1700	Returns 422 Unprocessable Entity	Pass
Automated test suite	Search project for test files	No dedicated test directory supplied	Not implemented

15. Deployment and Execution Procedure

15.1 Recommended Local Setup

Step 1 — Install dependencies:

```
pip install -r requirements.txt
```

Step 2 — Train the model (only needed once, or to regenerate artefacts):

```
python train_model.py
```

Step 3 — Terminal 1: Start the FastAPI backend:

```
uvicorn app:app --reload
```

Backend available at: http://localhost:8000 | Swagger docs: http://localhost:8000/docs

Step 4 — Terminal 2: Start the Streamlit frontend:

```
streamlit run frontend.py
```

Frontend available at: http://localhost:8501

Both processes must remain running simultaneously. The Streamlit frontend will display an error if the FastAPI backend is not running on port 8000.

15.2 Required Files at Runtime

File	Purpose
app.py	FastAPI backend — must be running via uvicorn.
frontend.py	Streamlit frontend — calls the backend API.
model/model.pkl	Trained scikit-learn Pipeline (StandardScaler + LinearRegression).
model/meta.json	Feature list, model name, metrics, all_model_metrics, feature stats.

16. Limitations and Technical Considerations

Limitation	Impact / Consideration
Dataset representativeness	No real-world geographic source documented. Results should be interpreted as performance on this specific project dataset.
Feature coverage	Important variables such as exact location, property condition, renovation history, school access, taxes, and market date are absent.
No feature engineering	House_Age, Total_Rooms, and Sqft_Per_Room are not computed, which limits the signal available to tree-based models.
Single split	Model selection is based on one 80/20 split. A different random state could produce a different ranking.
Model persistence compatibility	joblib artefacts can be sensitive to library versions. Use scikit-learn 1.4.2 (pinned in requirements.txt) to reproduce the saved model.
Two-process dependency	The Streamlit frontend requires the FastAPI backend to be running on port 8000. If the backend is down, all predictions fail with a connection error.
No production monitoring	There is no drift detection, model versioning, input logging, authentication, or automated retraining.

17. Future Enhancements

• Add Docker Compose to run both backend and frontend as containerised services.

• Add feature engineering (House_Age, Total_Rooms, Sqft_Per_Room) to improve model signal.

• Extend the model comparison to include Ridge Regression and XGBoost.

• Add hyperparameter tuning for Random Forest and Gradient Boosting via GridSearchCV or Optuna.

• Introduce cross-validation to obtain more stable model comparison results.

• Add calibrated prediction intervals based on residual analysis or conformal prediction.

• Add automated unit and integration tests for data loading, model inference, and API endpoints.

• Add logging, input validation, drift monitoring, and model versioning for production deployment.

• Replace the hardcoded dataset average ($618,861) in the gauge with a value fetched dynamically from the /model-info endpoint.

• Add an export function for prediction summaries and analytical charts.

18. Conclusion

The House Price Prediction System is a complete academic machine-learning web application that connects data preparation, model comparison, model persistence, a REST API backend, and an interactive web frontend. The project demonstrates the standard two-tier deployment pattern for ML systems: a FastAPI backend that owns the model and serves predictions via /predict, and a Streamlit frontend that consumes the API and renders results interactively.

On the supplied 1,000-record dataset, Linear Regression (within a StandardScaler Pipeline) achieved R^2 = 0.9984, MAE = $8,174.58, and RMSE = $10,071.48. The system successfully demonstrates the full regression lifecycle — from training and evaluation to API serving and frontend consumption — and is well-suited for academic presentation and further extension.

References

[1] FastAPI Documentation — https://fastapi.tiangolo.com/

[2] Streamlit Documentation — Get Started — https://docs.streamlit.io/get-started

[3] scikit-learn Documentation — Pipeline — https://scikit-learn.org/stable/modules/pipeline.html

[4] scikit-learn Documentation — Ensemble Methods — https://scikit-learn.org/stable/api/sklearn.ensemble.html

[5] Uvicorn Documentation — https://www.uvicorn.org/

[6] Project source code — train_model.py, app.py, frontend.py (local project files).

Appendix A — Project Structure

```
House_Price_Regression/
├── app.py                                  # FastAPI backend
├── frontend.py                             # Streamlit frontend
├── train_model.py                          # Model training
script
├── house_price_regression_dataset.csv      # Training dataset
(1,000 rows)
├── requirements.txt                        # Python dependencies
├── README.md
├── model/
│   ├── model.pkl          # Saved best model (scikit-learn
Pipeline)
│   └── meta.json          # Feature list, metrics, feature
stats
└── plots/
```

```
    └── feature_importance.png   # Feature importance chart
```

Appendix B — Key Implementation Excerpts

B.1 Model Definitions (train_model.py)

```
models = {
    "LinearRegression": Pipeline([
        ("scaler", StandardScaler()),
        ("model",  LinearRegression()),
    ]),
    "RandomForest": RandomForestRegressor(n_estimators=200,
random_state=42),
    "GradientBoosting": GradientBoostingRegressor(n_estimators=300,
learning_rate=0.05,
                                                  max_depth=5,
random_state=42),
}
```

B.2 FastAPI Prediction Endpoint (app.py)

```
@app.post("/predict", response_model=PredictionResponse)
def predict(house: HouseFeatures):
    input_df = pd.DataFrame([[
        house.Square_Footage, house.Num_Bedrooms,
house.Num_Bathrooms,
        house.Year_Built, house.Lot_Size, house.Garage_Size,
        house.Neighborhood_Quality,
    ]], columns=FEATURES)
    price = float(model.predict(input_df)[0])
    return PredictionResponse(predicted_price=round(price, 2),
model_used=meta["model_name"])
```

B.3 Frontend API Call (frontend.py)

```
payload = {
    "Square_Footage": square_footage, "Num_Bedrooms": num_bedrooms,
    "Num_Bathrooms": num_bathrooms, "Year_Built": year_built,
    "Lot_Size": lot_size, "Garage_Size": garage_size,
    "Neighborhood_Quality": neighborhood_quality,
}
result = requests.post("http://localhost:8000/predict",
json=payload).json()
price = result["predicted_price"]
```